

WEEK 1: PROJECT PLANNING AND STRATEGY FOR ML PIPELINE

Table of Contents

1. Introduction.....	1
2. Literature Review.....	3
2.1 Overview of Recommendation Systems.....	3
2.2 Collaborative Filtering.....	3
2.3 Content-Based Filtering.....	3
2.4 Hybrid Recommendation Systems.....	4
2.5 Evaluation Metrics.....	5
2.6 MLOps and Production ML.....	5
3. Problem Statement.....	6
3.1 Core Problem.....	6
3.2 Inputs.....	6
3.3 Potential Challenges.....	7
3.4 Expected Outcomes.....	7
4. Project Objectives.....	8
5. Scope and Deliverables.....	9
5.1 In-Scope.....	9
5.2 Out-of-Scope.....	9
5.3 Deliverables.....	9
6. Detailed Timeline with Daily Tasks.....	11
7. Key Milestones.....	16
8. Resource Requirements.....	18
8.1 Software and Libraries.....	18
8.2 Computational Resources.....	19
8.3 Data Resources.....	19
8.4 Human Resources.....	20
9. Risk Management.....	21
10. Strategic Roadmap for Implementation.....	26
10.1 Phased Implementation Strategy.....	26
10.2 Key Technical Decisions.....	30
10.3 Success Metrics.....	31

11. Success Criteria and Evaluation Metrics	34
11.1 Technical Success Criteria	34
11.2 Academic Evaluation Criteria	34
12. Conclusion	35
13. References.....	36

1. Introduction

The rapid growth of e-commerce platforms and digital content services has created an overwhelming abundance of choices for consumers, making it increasingly difficult for users to discover relevant products, movies, or content. Recommendation systems have emerged as a critical solution, enabling personalized user experiences that drive engagement, retention, and revenue. These systems analyze user behavior, preferences, and item characteristics to suggest items that a user is likely to find valuable.

Real-World Context:

- **Amazon** reports that 35% of its total sales come from its recommendation engine
- **Netflix** estimates that its recommendation system saves the company \$1 billion annually through reduced churn
- **Spotify's** Discover Weekly playlist, powered by hybrid recommendations, has over 40 million weekly active users

This Project:

This project aims to design and implement a comprehensive end-to-end Machine Learning pipeline for a Hybrid Recommendation System using the MovieLens 100K dataset. The system will combine collaborative filtering and content-based filtering techniques to leverage the strengths of both approaches while mitigating their individual limitations.

Dataset Details:

- 100,000 ratings from 943 users on 1,682 movies
- Rating scale: 1-5 (integer values)
- Time period: September 1997 - April 1998
- Sparsity: 93.7% (only 6.3% of possible user-item pairs have ratings)
- Contains user demographics (age, gender, occupation) and movie metadata (genres, release date)

Collaborative filtering excels at capturing user-item interaction patterns but suffers from the cold-start problem, while content-based filtering effectively handles new items but can create filter bubbles by recommending overly similar content.

The project will be executed over a 6-week period (August 14, 2026 – September 25, 2026) as part of a mandatory academic summer internship. The deliverables include a fully

functional ML pipeline, a deployed REST API, comprehensive documentation, and a detailed project report.

Significance:

By building a hybrid recommendation system, this project showcases the ability to:

- Integrate multiple ML techniques (SVD + TF-IDF + XGBoost)
- Handle large-scale data (100,000+ interactions)
- Implement production-grade APIs (FastAPI + Streamlit)
- Apply modern MLOps practices (MLflow + ZenML + Docker)

2. Literature Review

2.1 Overview of Recommendation Systems

Recommendation systems have become a cornerstone of modern digital platforms. The fundamental goal is to predict user preferences and suggest items that align with their interests.

Market Impact:

- **Amazon:** 35% of revenue from recommendations
- **Netflix:** 80% of watched content from recommendations
- **YouTube:** 70% of watch time from recommendations

2.2 Collaborative Filtering

Collaborative filtering (CF) operates on the principle that users who have agreed in the past will agree in the future.

Types of CF:

- **User-based CF:** Finds similar users based on rating patterns (Resnick et al., 1994)
- **Item-based CF:** Finds similar items based on user ratings (Sarwar et al., 2001)
- **Matrix Factorization:** Decomposes user-item matrix into latent factors; SVD and ALS are most effective (Koren et al., 2009)

Success Story:

The **Netflix Prize** (2006-2009) offered \$1 million to improve recommendation accuracy by 10%. The winning solution used an ensemble of **107 different CF models**, achieving 10.06% improvement through matrix factorization techniques.

Limitations:

- Cold-start problem (cannot recommend new items or to new users)
- Data sparsity (most user-item pairs have no ratings)
- Popularity bias (recommends only popular items)

2.3 Content-Based Filtering

Content-based filtering (CBF) recommends items based on similarity between item features and user profiles. It relies on item metadata such as descriptions, categories, and tags (Lops et al., 2011).

Techniques:

- **Feature Extraction:** TF-IDF (Term Frequency-Inverse Document Frequency)
- **Similarity Computation:** Cosine similarity between item vectors

Example:

If a user rates "The Matrix" (Action, Sci-Fi) highly, CBF will recommend other Action/Sci-Fi movies like "The Terminator" or "Inception."

Limitations:

- Overspecialization (filter bubble)
- Limited feature representation
- Requires rich item metadata

2.4 Hybrid Recommendation Systems

Hybrid systems combine multiple techniques to overcome individual limitations (Burke, 2002).

Approach	Description	Example
Weighted	Combine scores with weighted average	$\text{Score} = 0.6 \times \text{CF} + 0.4 \times \text{CBF}$
Switching	Switch based on context	Use CF for known users, CBF for new users
Feature Combination	Combine features into single model	Use user-item interactions + item metadata
Meta-learning	Learn optimal combination	Train XGBoost on CF and CBF features

Success Story:

Spotify's Discover Weekly uses a hybrid approach combining collaborative filtering (what similar users like), content-based filtering (audio features), and NLP (lyrics analysis). This generates 40 million+ personalized playlists weekly.

2.5 Evaluation Metrics

Metric	Formula	Why Used	Practical Example
Precision@k	Relevant in Top k / k	Measures recommendation quality	"Precision@10 = 30% means 3 of 10 recommended items are liked by user"
Recall@k	Relevant in Top k / Total Relevant	Measures coverage	"Recall@10 = 40% means we found 4 of 10 relevant items in Top 10"
NDCG@k	Discounted Cumulative Gain	Prioritizes top positions	"A relevant item at #1 is more valuable than at #10 (higher score)"
RMSE	Root Mean Square Error	Measures prediction accuracy	"RMSE = 0.85 means predictions are ~0.85 stars from actual ratings"

2.6 MLOps and Production ML

Modern ML projects require robust engineering practices (Sculley et al., 2015):

- **Experiment Tracking:** MLflow for logging parameters, metrics, and artifacts
- **Pipeline Orchestration:** ZenML for automating data processing, training, and deployment
- **Model Monitoring:** Detecting data drift and performance degradation
- **Explainability:** SHAP and LIME for interpretable recommendations

3. Problem Statement

3.1 Core Problem

E-commerce platforms and content streaming services face a significant challenge: users are overwhelmed by the sheer volume of available items, leading to decision paralysis, reduced engagement, and lower conversion rates.

Example:

- Amazon lists over 12 million products
- Netflix has 15,000+ titles
- Spotify has 100 million+ songs

Why Traditional Approaches Fail:

1. **Collaborative filtering** fails to recommend new items (cold-start problem) and performs poorly for users with limited interaction history.
2. **Content-based filtering** creates filter bubbles, recommending only items similar to what the user has already seen, reducing serendipity and discovery.

This Project's Solution:

Build a robust hybrid recommendation system that:

- Provides accurate, diverse, and personalized recommendations
- Handles cold-start scenarios using content-based fallback
- Achieves **15-20% improvement** in Precision@10 over individual approaches

3.2 Inputs

The system will ingest the following data:

Data Type	Description	Source
User-Item Interactions	user_id, item_id, rating (1-5), timestamp	MovieLens 100K (100,000 ratings)
Item Metadata	item_id, title, release_date, 19 genres	MovieLens 100K (1,682 movies)
User Demographics	user_id, age, gender, occupation, zip_code	MovieLens 100K (943 users)

3.3 Potential Challenges

Challenge	Description	Impact
Data Sparsity	Only 6.3% of possible user-item pairs have ratings	Reduces CF accuracy
Cold-Start	New users/items have no interaction history	Makes CF impossible for new entities
Scalability	Large datasets require efficient algorithms	Increases training and inference time
Explainability	Users need to trust recommendations	Black-box models reduce user adoption
Evaluation	Offline metrics may not correlate with online business metrics	May not capture real-world performance

Example of Sparsity:

In MovieLens 100K: $100,000 \text{ ratings} \div (943 \text{ users} \times 1,682 \text{ movies}) = 6.3\% \text{ density}$

3.4 Expected Outcomes

Outcome	Description	Target
Accurate Recommendations	High Precision@k and Recall@k	Precision@10 > 0.30
Cold-Start Coverage	Recommendations for new users/items	Coverage > 80%
Model Explainability	SHAP/LIME explanations	Actionable insights
Production API	REST API for real-time serving	Latency < 500ms
Reproducible Pipeline	Orchestrated ML pipeline	One-click execution

4. Project Objectives

Objective	Description	Priority
O1: Design and implement a hybrid recommendation system	Combine CF (SVD) and CBF (TF-IDF + Cosine Similarity)	High
O2: Build an end-to-end ML pipeline	Modular pipeline from data ingestion to deployment	High
O3: Implement production-grade API	FastAPI endpoint with low-latency predictions	High
O4: Integrate explainability	SHAP/LIME for recommendation transparency	Medium
O5: Apply MLOps best practices	MLflow tracking, ZenML orchestration, Docker	Medium
O6: Produce comprehensive documentation	Design decisions, implementation details, usage	High

5. Scope and Deliverables

5.1 In-Scope

- Data preprocessing and EDA (pandas, matplotlib, seaborn)
- Collaborative filtering (Surprise library, SVD)
- Content-based filtering (scikit-learn, TF-IDF)
- Hybrid system (Weighted + Meta-learner with XGBoost)
- Evaluation framework (Precision@k, Recall@k, NDCG@k, RMSE)
- Experiment tracking (MLflow)
- Pipeline orchestration (ZenML)
- REST API (FastAPI)
- UI demo (Streamlit)
- Documentation and project report

5.2 Out-of-Scope

- Deep learning models (neural collaborative filtering) – future work
- Real-time streaming data processing
- A/B testing in production
- Advanced personalization with user context (location, time, device)
- Large-scale distributed training

5.3 Deliverables

Deliverable	Format	Deadline
Project Plan (Revised)	DOCX, PDF	21-Aug-2026
Data Pipeline & EDA	Code + Report	27-Aug-2026
Collaborative Filtering Model	Code + Evaluation	3-Sep-2026
Content-Based Model	Code + Evaluation	10-Sep-2026
Hybrid System	Code + Evaluation	17-Sep-2026
MLOps Pipeline & API	Code + Documentation	24-Sep-2026
Final Internship Report	DOCX	30-Sep-2026
Final Presentation	PPT	30-Sep-2026

Deliverable	Format	Deadline
GitHub Repository	URL	30-Sep-2026

6. Detailed Timeline with Daily Tasks

Phase 1: Project Planning and Research (Week 1: Aug 14-20)

Day	Date	Task	Description	Effort (hrs)
Day 1	14-Aug	Literature Review	Read 5-8 papers on SVD, CF, CBF, hybrid systems	8
Day 2	15-Aug	Dataset Analysis	Download MovieLens, understand structure, calculate sparsity	4
Day 3	16-Aug	Tool Selection	Evaluate pandas, surprise, scikit-learn, ZenML, MLflow	6
Day 4	17-Aug	Problem Definition	Refine statement, define success criteria, write objectives	6
Day 5	18-Aug	Technical Design	System architecture, component diagrams, data flow	6
Day 6-7	19-20 Aug	Write Project Plan	Complete all sections, format, review, submit	12

Effort: ~42 hours | Deliverable: Project Plan (DOCX, PDF)

Phase 2: Data Engineering (Week 2: Aug 21-27)

Day	Date	Task	Description	Effort (hrs)
Day 1	21-Aug	Data Ingestion	Load u.data, u.item, u.user into pandas DataFrames	2
Day 2	22-Aug	Data Cleaning	Handle missing values, drop video_release_date, fill release_date	6

Day	Date	Task	Description	Effort (hrs)
Day 3	23-Aug	Exploratory Analysis	Visualize distributions, patterns, and trends	8
Day 4	24-Aug	Feature Engineering	Create user, movie, and temporal features	8
Day 5	25-Aug	Data Validation	Check data quality, verify transformations	4
Day 6-7	26-27 Aug	Documentation	Write EDA report, update GitHub	6

Effort: ~34 hours | Deliverable: Clean dataset, EDA report, data pipeline

Phase 3: Collaborative Filtering (Week 3: Aug 28 - Sep 3)

Day	Date	Task	Description	Effort (hrs)
Day 1	28-Aug	Baseline Model	Implement popularity-based recommender	4
Day 2	29-Aug	SVD Implementation	Train SVD using Surprise library, tune hyperparameters	6
Day 3-4	30-31 Aug	Hyperparameter Tuning	Grid search: n_factors (10-100), reg (0.01-0.1), lr (0.001-0.01)	8
Day 5	1-Sep	Evaluation	Compute RMSE, Precision@k, Recall@k, NDCG@k	6

Day	Date	Task	Description	Effort (hrs)
Day 6	2-Sep	Error Analysis	Analyze failure cases, document findings	4
Day 7	3-Sep	Documentation	Write CF model report, update GitHub	4

Effort: ~32 hours | Deliverable: CF model, evaluation report

Phase 4: Content-Based Filtering (Week 4: Sep 4-10)

Day	Date	Task	Description	Effort (hrs)
Day 1	4-Sep	Feature Extraction	TF-IDF on movie genres, descriptions	6
Day 2	5-Sep	Similarity Computation	Compute cosine similarity between items	4
Day 3	6-Sep	User Profiling	Create user profiles from item interactions	6
Day 4	7-Sep	Generate Recommendations	Evaluate CBF on test users	6
Day 5	8-Sep	Evaluation	Compare performance against CF baseline	4
Day 6-7	9-10 Sep	Documentation	Write CBF model report, update GitHub	6

Effort: ~32 hours | Deliverable: CBF model, evaluation report

Phase 5: Hybrid System (Week 5: Sep 11-17)

Day	Date	Task	Description	Effort (hrs)
Day 1	11-Sep	Weighted Hybrid	Combine CF and CB scores (weighted average: 0.3-0.7 range)	6
Day 2	12-Sep	Switch Hybrid	Implement switching for cold-start detection	4
Day 3-4	13-14 Sep	Meta-Learner Hybrid	Train XGBoost on CF and CBF features	8
Day 5	15-Sep	Evaluation	Compare hybrid performance vs. individual models	6
Day 6	16-Sep	Explainability	Integrate SHAP for recommendation explanations	4
Day 7	17-Sep	Documentation	Write hybrid system report, update GitHub	4

Effort: ~32 hours | Deliverable: Hybrid model, final evaluation report

Phase 6: MLOps and Deployment (Week 6: Sep 18-25)

Day	Date	Task	Description	Effort (hrs)
Day 1	18-Sep	Pipeline Orchestration	Build ZenML pipeline with modular steps	6
Day 2	19-Sep	Experiment Tracking	Set up MLflow for parameter and artifact logging	4
Day 3	20-Sep	Model Serialization	Save optimized model artifacts (pickle, joblib)	2

Day	Date	Task	Description	Effort (hrs)
Day 4-5	21-22 Sep	REST API Development	Build FastAPI endpoint with request/response models	8
Day 6	23-Sep	Streamlit UI	Create interactive demo with user input and recommendations	4
Day 7	24-Sep	Documentation	Finalize all documentation, update GitHub	4

Effort: ~28 hours | Deliverable: Working API, UI demo, Docker container

Phase 7: Finalization (Week 6 - Buffer: Sep 26-30)

Day	Date	Task	Description	Effort (hrs)
Day 1-3	26-28 Sep	Internship Report	Compile complete report with all sections	12
Day 4	29-Sep	PPT Preparation	Create presentation slides with visuals	6
Day 5	30-Sep	GitHub Cleanup	Finalize code repository, tag final version	4
Day 6	30-Sep	Viva Preparation	Review technical concepts, practice explanations	3

Effort: ~25 hours | Deliverable: Internship Report, PPT, GitHub URL

7. Key Milestones

Milestone	Target Date	Deliverable	Success Criteria	Status
M1: Project Plan Approved	21-Aug-2026	Project Plan DOCX (Revised)	All sections complete, TOC auto-generated, evaluator score >70/100	Pending
M2: Data Pipeline Complete	27-Aug-2026	Clean dataset, EDA report	3 datasets cleaned, 5+ visualizations, master_dataset.csv saved	Pending
M3: Baseline CF Model	3-Sep-2026	CF model + evaluation	RMSE < 1.0, Precision@10 > 0.25	Pending
M4: Content-Based Model	10-Sep-2026	CBF model + evaluation	RMSE < 1.0, Precision@10 > 0.22	Pending
M5: Hybrid System Complete	17-Sep-2026	Hybrid model + final evaluation	15% improvement over baseline, Precision@10 > 0.30	Pending

Milestone	Target Date	Deliverable	Success Criteria	Status
M6: API Deployed	25-Sep-2026	FastAPI endpoint, UI demo	Latency < 500ms, 3+ test cases pass	Pending
M7: Final Submission	30-Sep-2026	Report, PPT, GitHub URL	All deliverables uploaded, README updated	Pending

8. Resource Requirements

8.1 Software and Libraries

Category	Tools/Libraries	Version	Purpose
Language	Python	3.9+	Core programming
Data Processing	pandas	1.5.3	Data manipulation
	numpy	1.24.0	Numerical operations
Data Visualization	matplotlib	3.7.0	Charts and plots
	seaborn	0.12.2	Statistical visualizations
	plotly	5.14.0	Interactive plots
Machine Learning	scikit-learn	1.2.2	TF-IDF, cosine similarity
	xgboost	1.7.3	Meta-learner hybrid
Collaborative Filtering	surprise	1.1.3	SVD, model training
	implicit	0.6.1	ALS alternative
Experiment Tracking	mlflow	2.2.2	Logging parameters, metrics
Pipeline Orchestration	zenml	0.20.0	End-to-end ML pipeline
API Development	fastapi	0.95.0	REST API serving
	uvicorn	0.21.1	ASGI server
UI Demo	streamlit	1.22.0	Interactive user interface

Category	Tools/Libraries	Version	Purpose
Explainability	shap	0.41.0	Model interpretability
	lime	0.2.0.1	Local explanations
Containerization	docker	23.0.0	Application packaging
Version Control	git	2.40.0	Code management
	github	N/A	Repository hosting

8.2 Computational Resources

Resource	Specification	Purpose	Cost
Development Machine	16GB RAM, 4+ cores, Windows 11	Local development, small experiments	Existing
Cloud GPU	Google Colab Free / Kaggle	GPU-accelerated training (XGBoost)	Free
Cloud VM (Optional)	AWS EC2 / GCP t2.medium	Deployment testing	~\$0.05/hr
Storage	GitHub (1GB free)	Code and small datasets	Free

Recommendation: Use Google Colab for XGBoost training if local machine is slow.

8.3 Data Resources

Dataset	Size	Source	License	Download Link
MovieLens 100K	100,000 ratings, 1,682 movies,	GroupLens	Research -friendly	https://grouplens.org/datasets/movielens/100k/

Dataset	Size	Source	License	Download Link
	943 users			
MovieLens 1M (Optional)	1,000,000 ratings	GroupLens	Research -friendly	https://grouplens.org/datasets/movielens/1m/

8.4 Human Resources

Role	Responsibility	Person	Contact
ML Engineer	Full development and implementation	Tirumala Karthik	GitHub: TirumalaKarthik
Mentor/Supervisor	Guidance and evaluation	Keerthi	Via internship portal

9. Risk Management

Risk	Prob.	Impact	Step-by-Step Mitigation Strategy	Contingency Plan
R1: Data Quality Issues	Medium	High	Step1: Run <code>df.isnull().sum()</code> daily to track missing values. Step 2: Impute missing numeric values with column median. Step 3: Impute missing categorical with mode. Step 4: Document all data quality issues in Jupyter Notebook.	Step1: Use <code>sklearn.datasets.make_classification</code> to generate synthetic data. Step 2: Download MovieLens 1M as alternative. Step 3: Contact mentor if both fail.
R2: Model Underperformance	Medium	High	Step 1: Build popularity-based baseline (target RMSE ~ 1.0). Step 2: Use 5-fold cross-validation. Step3: Incrementally add features and track improvement. Step 4: If no improvement after 3	Step 1: Use linear regression as simpler model. Step 2: Use KNN as another alternative. Step 3: Ensemble multiple simple models for better performance.

Risk	Prob.	Impact	Step-by-Step Mitigation Strategy	Contingency Plan
			iterations, switch algorithms.	
R3: Computational Limitations	Medium	Medium	Step 1: Profile code using %timeit to identify bottlenecks. Step 2: Vectorize operations using NumPy. Step 3: Use Google Colab free GPU for XGBoost training. Step 4: Optimize hyperparameter search using Bayesian optimization (Optuna).	Step 1: Reduce dataset to 50,000 samples. Step 2: Use precomputed features saved to CSV. Step3: Use joblib to cache intermediate results.
R4: Integration Complexity	Medium	Medium	Step 1: Design interfaces between components before coding. Step 2: Test each component in	Step 1: Simplify by reducing number of components. Step 2: Use Flask instead of FastAPI if complex.

Risk	Prob.	Impact	Step-by-Step Mitigation Strategy	Contingency Plan
			isolation with unit tests. Step 3: Use ZenML for structured pipeline. Step 4: Document all API endpoints with FastAPI's auto-documentation.	Step 3: Use proven libraries over custom implementations.
R5: Time Constraints	High	High	Step 1: Daily progress tracking using checklist. Step 2: Prioritize MVP: CF + CBF + Basic API. Step 3: Front-load complex work (Week 1-3). Step 4: Use buffer week for finalization.	Step 1: Cut Streamlit UI and use only API. Step 2: Use pre-built SVD from Surprise library. Step 3: Skip SHAP explainability and add later.
R6: Dataset Size	Low	Medium	Step 1: Start with MovieLens 100K for prototyping.	Step 1: Use 10% random subset of data.

Risk	Prob.	Impact	Step-by-Step Mitigation Strategy	Contingency Plan
			Step 2: Scale to MovieLens 1M if ahead of schedule. Step 3: Use dask if memory issues arise.	Step 2: Use preprocessed/subsampled data.
R7: Deployment Issues	Medium	Medium	Step 1: Test API locally with curl and Postman. Step 2: Use Docker for containerization. Step 3: Test in a clean environment before production.	Step 1: Use local API only (no cloud deployment). Step 2: Use Hugging Face Spaces for quick deployment.
R8: Viva Preparation	Medium	High	Step 1: Document all decisions in report. Step 2: Create presentation slides early (Week 5). Step 3: Practice explanations weekly.	Step 1: Create cheat sheet with key concepts. Step 2: Record a practice viva session. Step 3: Review all code and notebooks

Risk	Prob.	Impact	Step-by-Step Mitigation Strategy	Contingency Plan
			Step 4: Prepare answers for top 10 likely questions.	

10. Strategic Roadmap for Implementation

10.1 Phased Implementation Strategy

Phase 1: Foundation (Week 1-2)

Element	Details
Goal	Establish solid foundation with thorough planning and data understanding
Critical Success Factor	Clean, well-understood data with <5% missing values
Key Decision	Dataset selection (MovieLens 100K) and architecture (Hybrid CF + CBF)
Exit Criteria	Project plan approved, data loaded and cleaned, EDA complete
Deliverables	Project plan DOCX, clean dataset, EDA report

Phase 2: Core Algorithms (Week 3-4)

Element	Details
Goal	Build individual components (CF and CBF) independently
Critical Success Factor	Strong baseline performance (RMSE < 1.0, Precision@10 > 0.25)
Key Decision	Select best-performing CF (SVD vs. ALS) and CBF configurations
Exit Criteria	Both models trained, evaluated, and documented
Deliverables	CF model, CBF model, evaluation reports

Phase 3: Integration (Week 5)

Element	Details
Goal	Hybridize the two approaches for 15%+ improvement

Element	Details
Critical Success Factor	Hybrid outperforms individual models (Precision@10 > 0.30)
Key Decision	Hybridization strategy (Weighted vs. Meta-learner with XGBoost)
Exit Criteria	Hybrid model validated and documented
Deliverables	Hybrid model, final evaluation report

Phase 4: Productionization (Week 6)

Element	Details
Goal	Package the system for deployment
Critical Success Factor	Functional API with reasonable latency (<500ms)

Element	Details
Key Decision	Deployment platform (local vs. cloud)
Exit Criteria	API working, UI demo functional, Docker container ready
Deliverables	Working API, UI demo, Docker container

Phase 5: Finalization (Week 6 Buffer)

Element	Details
Goal	Complete documentation and prepare for evaluation
Critical Success Factor	Comprehensive report and viva readiness
Key Decision	N/A

Element	Details
Exit Criteria	Report completed, PPT ready, viva practiced
Deliverables	Internship Report, PPT, GitHub URL

10.2 Key Technical Decisions

Decision Point	Options	Decision	Rationale
CF Algorithm	SVD, ALS, NMF	SVD	Simplicity, well-documented, good performance
CBF Vectorization	TF-IDF, Word2Vec, BERT	TF-IDF	Simplicity, interpretability, works well with short text
Hybridization	Weighted, Switching, Meta-learner	Weighted + Meta-learner	Weighted is simple; Meta-learner for advanced comparison

Decision Point	Options	Decision	Rationale
Pipeline Framework	ZenML, Kedro, Custom	ZenML	Good for beginners, integrates with MLflow
API Framework	FastAPI, Flask	FastAPI	Modern, fast, auto-documentation
UI Demo	Streamlit, Gradio	Streamlit	Simple, Python-native, good for demos

10.3 Success Metrics

Metric	Target	How to Measure	Why This Target
Precision@10	>0.30	<code>precision_at_k(y_true, y_pred, k=10)</code>	Industry-standard; demonstrates recommendation relevance

Metric	Target	How to Measure	Why This Target
Recall@10	>0.20	recall_at_k(y_true, y_pred, k=10)	Ensures coverage of relevant items
NDCG@10	>0.50	ndcg_at_k(y_true, y_pred, k=10)	Accounts for ranking quality
RMSE	<0.90	sqrt(mean_squared_error(y_true, y_pred))	Accurate rating predictions
Cold-Start Coverage	>80%	Percentage of new users/items with recommendations	Users always get recommendations
API Latency	<500ms	Measure response time with timeit	Usable in production

Metric	Target	How to Measure	Why This Target
Model Improvement	>15% over baseline	Compare hybrid vs. baseline Precision@10	Hybrid system must demonstrate value

11. Success Criteria and Evaluation Metrics

11.1 Technical Success Criteria

1. **Functional System:** Complete pipeline from data ingestion to API serving recommendations
2. **Model Performance:** Hybrid model outperforms both CF and CBF individually by at least 15% in Precision@10
3. **Cold-Start Handling:** System provides recommendations for users with <5 interactions
4. **Reproducibility:** Full pipeline can be run from scratch to reproduce results
5. **Documentation:** Clear, comprehensive documentation covering all aspects

11.2 Academic Evaluation Criteria

Criterion	Description	Weight
Project Plan	Clarity, completeness, feasibility	10%
Technical Implementation	Code quality, functionality, innovation	30%
Internship Report	Structure, clarity, depth of content	30%
PPT Presentation	Visual design, clarity, flow	10%
Viva Performance	Conceptual understanding, articulation	20%

12. Conclusion

This project plan outlines a comprehensive 6-week implementation of a **Hybrid Recommendation System**, designed to meet the requirements of the mandatory academic summer internship. The project leverages a strong theoretical foundation in machine learning algorithms, applying them to build a practical, production-ready system.

Key Strengths:

- Real-world problem with measurable business impact
- Specific, achievable targets (Precision@10 > 0.30, RMSE < 0.90)
- Comprehensive risk management with clear contingency plans
- Professional MLOps practices (MLflow, ZenML, Docker)
- Clear evaluation metrics with practical examples

The phased approach ensures systematic progress, with clear milestones and deliverables at each stage. The hybrid system addresses the core challenges of recommendation systems—cold-start and data sparsity—by combining collaborative and content-based filtering.

Upon completion, this project will serve as a strong portfolio piece, demonstrating end-to-end ML engineering capabilities, theoretical understanding, and the ability to deliver a functional product with professional documentation.

Next Steps:

1. Begin data engineering (Week 2) immediately after project plan approval
2. Follow the detailed daily schedule
3. Maintain regular GitHub commits
4. Weekly mentor check-ins for feedback

13. References

1. Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331-370.
2. Herlocker, J. L., Konstan, J. A., Terveen, L. G., & Riedl, J. T. (2004). Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1), 5-53.
3. Koren, Y. (2009). The BellKor solution to the Netflix Grand Prize. *Netflix Prize Documentation*.
4. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30-37.
5. Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In *Recommender Systems Handbook* (pp. 73-105). Springer.
6. Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. *Proceedings of CSCW*, 175-186.
7. Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. *Proceedings of WWW*, 285-295.
8. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., ... & Young, M. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2503-2511.
9. Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1), 1-38.